.

🔵 **UNIT 1 – B: Fourier Transform for Image Filtering**

✅ **1. Aim**

To apply Fourier Transform on an image and perform filtering to remove noise or smooth the image.

🧠 **2. Concept (VERY SIMPLE)**

First understand this clearly:

👉 Normal image = **pixels (space domain)**
👉 Fourier Transform = converts image into **frequency domain**

Think like this:

- Smooth areas → **low frequency**
- Edges/noise → **high frequency**

👉 What we do:

1. Convert image → frequency
2. Remove unwanted frequencies (noise)
3. Convert back → image
4.

📜 **3. Algorithm (WRITE THIS IN EXAM)**

1. Read the input image in grayscale
2. Apply 2D Fourier Transform
3. Shift zero frequency to center
4. Create a filter mask (low-pass filter)
5. Multiply mask with frequency image
6. Apply inverse Fourier Transform
7. Convert result to magnitude form
8. Display filtered image

💻 4. Clean Code (NO COMMENTS)

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("input.jpg", 0)

dft = np.fft.fft2(img)
dft_shift = np.fft.fftshift(dft)

rows, cols = img.shape
mask = np.zeros((rows, cols), np.uint8)

crow, ccol = rows//2, cols//2
cv2.circle(mask, (ccol, crow), 50, 1, -1)

filtered = dft_shift * mask

ifft_shift = np.fft.ifftshift(filtered)
img_back = np.fft.ifft2(ifft_shift)
img_back = np.abs(img_back)

plt.imshow(img_back, cmap='gray')
plt.show()
```

🟢 UNIT 2 – A: Basic Image Operations

✅ 1. Aim

To perform basic image operations like reading, writing, conversion, and complement of an image.

🧠 2. Concept (VERY SIMPLE)

These are the **most basic steps in image processing**.

👉 Operations:

- **Read Image** → load image into program
- **Write Image** → save image
- **Conversion** → color → grayscale
- **Complement** → negative image

🔹 Key Idea

Pixel values range:

- 0 → black
- 255 → white

👉 Complement formula:

$$s = 255 - r$$

Meaning:

- black → white
- white → black

📜 3. Algorithm (WRITE THIS)

1. Read input image
2. Convert image to grayscale
3. Apply threshold to create binary image

4. Compute complement (negative image)
5. Save output images
6. Display all images

💻 4. Clean Code

```
import cv2

img = cv2.imread("input.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

_, binary = cv2.threshold(gray, 120, 255, cv2.THRESH_BINARY)

negative = 255 - gray

cv2.imwrite("gray.jpg", gray)
cv2.imwrite("binary.jpg", binary)
cv2.imwrite("negative.jpg", negative)

cv2.imshow("original", img)
cv2.imshow("gray", gray)
cv2.imshow("binary", binary)
cv2.imshow("negative", negative)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

UNIT 3 – A: Morphological Operations

✅ 1. Aim

To apply morphological operations like erosion, dilation, opening, and closing on an image.

🧠 2. Concept (VERY SIMPLE)

👉 These operations work on **binary images (0 and 255)**

👉 They modify the **shape of objects**

🔹 Core Idea

We use something called a **kernel (small matrix)**

It moves over the image and changes pixels.

🔸 Operations

1. Erosion

- Removes pixels from object
- Object becomes smaller

2. Dilation

- Adds pixels
- Object becomes bigger

3. Opening

- Erosion → Dilation
- Removes noise

4. Closing

- Dilation → Erosion
- Fills gaps

📜 3. Algorithm (WRITE THIS)

1. Read input image
2. Convert image to grayscale
3. Apply threshold to get binary image
4. Create structuring element (kernel)
5. Apply erosion operation
6. Apply dilation operation
7. Apply opening operation
8. Apply closing operation
9. Display results

💻 4. Clean Code

```
import cv2
import numpy as np

img = cv2.imread("input.jpg", 0)

_, binary = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)

kernel = np.ones((5,5), np.uint8)

erosion = cv2.erode(binary, kernel)
dilation = cv2.dilate(binary, kernel)
opening = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)
closing = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, kernel)

cv2.imshow("binary", binary)
cv2.imshow("erosion", erosion)
cv2.imshow("dilation", dilation)
cv2.imshow("opening", opening)
cv2.imshow("closing", closing)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

UNIT 4 – A: Texture Analysis and Synthesis

✅ 1. Aim

To analyze texture features of an image and generate (synthesize) a similar texture.

🧠 2. Concept (VERY SIMPLE)

🔹 What is Texture?

Texture = **repeated patterns** in an image

Examples:

- grass
- cloth
- sand

🔸 Two Parts

1. Texture Analysis

👉 Extract features (numbers) that describe texture

We use **GLCM (Gray Level Co-occurrence Matrix)**

It checks:
👉 "How often pixel pairs occur"

From this we get:

- **Contrast** → intensity difference
- **Energy** → uniformity
- **Homogeneity** → smoothness
- **Correlation** → relationship

2. Texture Synthesis

👉 Create new image using same texture pattern

Simple method:

- Pick random patches
- Combine them

📜 3. Algorithm (WRITE THIS)

Texture Analysis:

1. Read input image
2. Convert to grayscale
3. Compute GLCM matrix
4. Extract features (contrast, energy, etc.)
5. Display values

Texture Synthesis:

6. Create empty output image
7. Select random patches from input
8. Place patches in output
9. Display synthesized image

💻 4. Clean Code

```
import cv2
import numpy as np
from skimage.feature import graycomatrix, graycoprops

img = cv2.imread("input.jpg", 0)

glcm = graycomatrix(img, [1], [0], 256, symmetric=True, normed=True)

contrast = graycoprops(glcm, 'contrast')[0,0]
energy = graycoprops(glcm, 'energy')[0,0]
homogeneity = graycoprops(glcm, 'homogeneity')[0,0]
correlation = graycoprops(glcm, 'correlation')[0,0]

print(contrast, energy, homogeneity, correlation)
```

UNIT 5 – B: 2D DFT and DCT

✅ 1. Aim

To apply 2D Discrete Fourier Transform (DFT) and Discrete Cosine Transform (DCT) on an image and analyze the result.

🧠 2. Concept (VERY SIMPLE)

🔹 What is DFT?

DFT converts image → **frequency domain** using:

- sine + cosine

👉 Same idea as Unit 1 (FFT)

🔹 What is DCT?

DCT converts image → frequency but:

- uses only cosine
- more efficient

👉 Used in:

- **JPEG compression**

🔥 Key Difference

DFT	DCT
Uses sine + cosine	Uses only cosine
Complex output	Real output
Less efficient	More efficient

📜 3. Algorithm (WRITE THIS)

1. Read input image in grayscale
2. Apply 2D DFT using FFT
3. Shift frequency to center
4. Compute magnitude spectrum
5. Normalize image for DCT
6. Apply DCT
7. Display results

💻 4. Clean Code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("input.jpg", 0)

dft = np.fft.fft2(img)
dft_shift = np.fft.fftshift(dft)
magnitude = np.log(np.abs(dft_shift) + 1)

img_float = np.float32(img) / 255.0
dct = cv2.dct(img_float)

plt.subplot(1,2,1)
plt.imshow(magnitude, cmap='gray')

plt.subplot(1,2,2)
plt.imshow(dct, cmap='gray')

plt.show()
```